

claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_1c5f1f5c-9ba\agent-a992184849231a50e.jsonl`
- SHA-256: `c07b5b5b850e504aa2b31c242e9d417f61a851e9b72a37601238360ac3c1caf6`
- Source modified: `2026-06-20T08:08:10+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_1c5f1f5c-9ba`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`
```

Transcript

1. user (2026-06-20T08:05:13.020Z)

You are the lead designer consolidating three independent designs for KhelSutra's interactive per-rally selection feature into one recommendation, plus a tracked design-doc draft.

RESEARCH (the real engine ? treat as ground truth):

```
{
  "contract": {
    "tables": [
      {
        "name": "videos",
        "purpose": "One row per ingested video file. Parent of all segments (FK cascade). Status is the ingest verdict (processed/failed). user_id (v4) is NULL on local installs.",
        "columns": [
          "id TEXT PRIMARY KEY (= MD5 of the whole file, compute_video_id)",
          "filepath TEXT NOT NULL",
          "processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP",
          "status TEXT NOT NULL (e.g. 'processed' | 'failed')",
          "validation_results TEXT (JSON list of ValidationResult dicts)",
          "user_id TEXT (NULLABLE, added v4; NULL = local install)"
        ]
      },
      {
        "name": "play_segments",
        "purpose": "CONFIRMED rallies ? the table a per-rally selection UI consumes today. /api/compile and /api/query read here. One row per accepted rally.",
        "columns": [
          "id INTEGER PK AUTOINCREMENT (the segment_id the UI selects/stitches)",
          "video_id TEXT NOT NULL (FK videos.id ON DELETE CASCADE)",
          "start_time REAL NOT NULL (seconds)",
          "end_time REAL NOT NULL (seconds)",
          "duration REAL NOT NULL (= end-start, computed at insert)",
          "server TEXT (NULLABLE; only the motion segmenter sets it, and it is FABRICATED/random)",
          "receiver TEXT (NULLABLE; same ? motion-only, fabricated)",
          "metadata TEXT (extensible JSON; gemini writes {shot_count, shot_count_confidence, detector}; motion writes {shot_count, intensity, avg_speed_kmh} all random)"
        ]
      },
      {
        "name": "candidate_segments",
        "purpose": "RAW pre-confirmation detections (detector-agnostic), persisted by the hybrid segmenters only. Holds the human-correction-loop columns (human_label/notes/confirmed_segment_id) that a review UI would write ? but NO endpoint writes them yet.",
        "columns": [
```

```

        "id INTEGER PK AUTOINCREMENT",
        "video_id TEXT NOT NULL (FK videos.id ON DELETE CASCADE)",
        "detector TEXT NOT NULL (e.g. wasb_hybrid/yolo_hybrid/fusion key)",
        "chunk_index INTEGER (NULLABLE)",
        "start_time REAL NOT NULL",
        "end_time REAL NOT NULL",
        "duration REAL NOT NULL",
        "confidence REAL (NULLABLE)",
        "stats TEXT (detector-specific JSON metrics: peak/mean velocity, active_frame_count,
net_crossings, etc.)",
        "detection_params TEXT (JSON of the detector knobs used)",
        "confirmed_segment_id INTEGER (NULLABLE FK play_segments.id ON DELETE SET NULL ? link
to the confirmed rally)",
        "human_label TEXT (NULLABLE ? correction-loop label, UNUSED by any code path today)",
        "notes TEXT (NULLABLE ? correction-loop notes, UNUSED today)",
        "created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP",
        "user_id TEXT (NULLABLE, added v4)"
    ]
},
{
    "name": "events",
    "purpose": "Sub-rally events keyed to a play_segment. ONLY the motion segmenter writes
these, and every value is RANDOM/fabricated (not ground truth). Gemini/hybrid paths write NO
events.",
    "columns": [
        "id INTEGER PK AUTOINCREMENT",
        "segment_id INTEGER NOT NULL (FK play_segments.id ON DELETE CASCADE)",
        "timestamp REAL NOT NULL",
        "type TEXT NOT NULL ('serve' | 'smash' | 'foul' | 'winner')",
        "player TEXT (NULLABLE; from a fabricated player_pool)",
        "details TEXT (JSON; e.g. {speed_kmh}, {type})"
    ]
},
{
    "name": "jobs",
    "purpose": "Async job store (v2/v3). One row per /api/process submission. status =
EXECUTION state; the pipeline verdict lives inside result JSON. Not rally data, but the UI polls
GET /api/jobs/{id} for progress.",
    "columns": [
        "id TEXT PK (uuid hex)",
        "video_path TEXT NOT NULL",
        "video_id TEXT (NULLABLE)",
        "segmenter TEXT",
        "player_pool TEXT (JSON)",
        "max_frames INTEGER",
        "status TEXT NOT NULL DEFAULT 'queued' (queued|running|waiting|done|failed)",
        "progress TEXT",
        "error TEXT",
        "result TEXT (JSON full response dict incl. play_segments + report paths)",
        "created_at/started_at/finished_at TIMESTAMP",
        "policy TEXT (v3 JSON ExecutionPolicy)",
        "next_poll_at TIMESTAMP (v3)"
    ]
},
{
    "name": "user_models",
    "purpose": "Per-user model store (v5). Personalization groundwork ONLY ? nothing on the
served path reads it. Not consumed by a rally-selection UI.",
    "columns": [
        "id, user_id, version, baseline_version, feature_schema_hash, fusion_config(JSON),
classifier_json(JSON), promotion_evidence(JSON), n_videos_at_fit, is_active, created_at"
    ]
}
},
"rally_fields_today": [

```

```

    "play_segments.id ? the integer the UI selects and passes to /api/compile as segment_ids
(backend/infrastructure/database.py:125-135, backend/main.py:102-105,386)",
    "start_time / end_time / duration (REAL seconds) ? the interval the reel is cut from;
duration computed at insert as end-start (database.py:127-129,347)",
    "video_id ? links the rally to its source file; /api/compile resolves videos.filepath as
the raw source to cut (main.py:344,393; database.py:418-425)",
    "metadata JSON.shot_count + shot_count_confidence ? gemini (default) persists these from
Gemini's shuttle_exchange_count + confidence; falls back to max(3,int(dur/0.8)) when absent
(gemini.py:470-485)",
    "metadata JSON.detector ? the model_name string (e.g. 'gemini-2.5-flash')
(gemini.py:484)",
    "server / receiver ? present in schema and returned by query, but ONLY the 'motion'
segmenter sets them and they are random.sample fabrications, NOT real player identity
(motion.py:191,201; database.py:130-131)",
    "events (type/player/timestamp/details) ? serve/smash/foul/winner; ONLY motion writes them
and ALL values are random.* fabrications, queryable via /api/query event_type filter
(motion.py:177-252; database.py:451-453)",
    "candidate_segments.stats JSON (hybrid paths only) ? per-window peak/mean velocity,
active_frame_count, net_crossings; raw detector metrics, not surfaced by /api/query
(database.py:362-383)",
    "candidate_segments.confidence ? REAL, persisted by add_candidate_segment but currently
passed as None by the trajectory hybrids (database.py:163,362-383)"
],
    "rally_fields_roadmap": [
        "shot_type (smash/clear/drop/drive/net) per shot ? NO detection or column exists;
events.type only carries motion's fabricated serve/smash/foul/winner labels, never produced by
gemini/hybrid paths (motion.py:228-239)",
        "player identity per rally/shot ? NO real detection. play_segments.server/receiver and
events.player are random.sample picks from a hardcoded pool in the motion demo only;
PersonDetector tracks anonymous chunk-local ByteTrack IDs, never named players
(motion.py:25-26,191; person_detector.py make_tracker)",
        "match score / point score per rally ? ABSENT everywhere. No score/game/set column in any
table; grep for score/game_index/set_index in backend finds only eval-F1 'score' and a
detector_score_threshold config knob, never a match score (database.py whole file;
config/models.py:126)",
        "game/set index (which game of the match a rally belongs to) ? NO column and no detection;
rallies are a flat per-video list with no game/set grouping (database.py:122-135)",
        "confidence on confirmed rallies ? play_segments has no confidence column; only
candidate_segments.confidence exists and the live trajectory hybrids pass None into it
(database.py:122-135 vs 362-383)",
        "human correction state (accepted/rejected/nudged) ? columns EXIST
(candidate_segments.human_label, notes, confirmed_segment_id) but are dead: no API endpoint or
code path writes human_label/notes; UI_PROPOSAL B6 PATCH /api/segments/{id} is proposed, not
built (database.py:166-168; docs/UI_PROPOSAL.md:59)"
    ],
    "reel_compose": {
        "entrypoint": "POST /api/compile -> backend/main.py:342-396 (compile_highlights) ->
backend/pipeline/stitcher/compiler.py:303 VideoCompiler.compile_highlights. Batch/CLI variants:
tools/generate_highlights.py and main.py --trim-start/--trim-end both call the same
VideoCompiler.",
        "inputs": "CompileRequest = {video_id: str, segment_ids: List[int], output_filename: str}
(main.py:102-105). The endpoint loads videos.filepath via db.get_video, fetches ALL
play_segments via query_play_segments, filters to the requested segment_ids, applies the
stitching.min_shot_count floor (segments without a known shot_count are exempt), then extracts
time_pairs = [(start_time,end_time)] (main.py:344-386).
VideoCompiler.compile_highlights(raw_video_path, segments:[(start,end)], output_filename) is the
underlying contract ? it takes the RAW source path + interval pairs, NOT segment ids
(compiler.py:303).",
        "outputs": "A single MP4 written to output_dir/<safe output_filename>; the endpoint
returns {status:'success', filepath: out_path} (main.py:393-394). The browser then downloads it
via GET /api/highlights/{video_id}/{filename} (FileResponse stream; main.py:307-328). There is
NO summary/JSON sidecar emitted by compile ? the per-video observability report is a separate
artifact emitted during processing, not at compile time.",
        "notes": "Pipeline: _merge_overlapping (sort + collapse overlapping/abutting/duplicate
ranges so a rally never plays twice; drops end<=start) -> per-clip ffmpeg trim with

```

```

begin_padding/end_padding (default 0.5/0.5 from StitchingConfig) RE-ENCODED libx264 superfast
crf22 + aac, optional drawtext/logo watermark baked per-clip -> concat demuxer with -c copy
(clips are codec-identical so concat stream-copies). Out-of-range/invalid segments are skipped
with a logged summary; if ALL skip it raises. Selection is purely by segment_ids the caller
supplies ? the backend has no 'select best N' logic."
},
"playback": {
  "proxy_available": "NO. There is no low-res proxy video generated or stored for UI
scrubbing. The only downscaling is transient: gemini.py re-encodes/downscales upload chunks to
indexing.ai_chunk_scale_width (default 1280) purely to fit the provider upload cap, and those
chunk files are deleted immediately after analysis (gemini.py:189-235,531). No proxy/preview MP4
is persisted anywhere (grep for proxy/preview/low-res finds only docs + the chunk-scale knob).",
  "frame_access": "NO per-rally thumbnails or extracted frames exist for the UI. The only
frame extraction is wasb_infer.py::extract_frames
(backend/pipeline/detectors/wasb_infer.py:547-571) which decodes the whole video to index-named
PNGs (00000000.png ...) inside the WSL detector cache as a transient input to the GPU shuttle
detector; it is deleted on a successful run by the cache-hygiene logic (keep_frames default
false) and is never keyed to rallies or served. There is no thumbnail column and no /api/frames
endpoint.",
  "notes": "A per-rally clip UI today has two options, both requiring the RAW source file:
(1) play the single compiled reel after /api/compile + download via GET
/api/highlights/{video_id}/{filename}; (2) play the full source and seek to
play_segments.start_time/end_time (the times are exact seconds). There is NO endpoint that
streams an arbitrary [start,end] span or a single rally clip, and NO endpoint that serves the
original source video ? only compiled reels under output_dir are downloadable (main.py:307-328).
The current static UI (backend/api/static/app.js) has NO <video> element at all: it renders
rally cards, lets the user check segment_ids, compiles, and offers a download link; it also
reads data.rallies which is a known bug (the API returns play_segments), per UI_PROPOSAL B5
(app.js:344,378; UI_PROPOSAL.md:58). A real review UI would need to either serve the source file
or add a per-span clip/proxy endpoint ? neither exists."
},
"file_refs": [
  "backend/infrastructure/database.py:109-148 (base tables: videos, play_segments, events)",
  "backend/infrastructure/database.py:122-135 (play_segments columns)",
  "backend/infrastructure/database.py:137-148 (events columns)",
  "backend/infrastructure/database.py:150-178 (v1 candidate_segments + columns)",
  "backend/infrastructure/database.py:166-168 (human_label, notes, confirmed_segment_id
correction-loop columns)",
  "backend/infrastructure/database.py:180-205 (v2 jobs table)",
  "backend/infrastructure/database.py:207-217 (v3 jobs.policy/next_poll_at)",
  "backend/infrastructure/database.py:219-237 (v4 user_id on videos + candidate_segments)",
  "backend/infrastructure/database.py:239-285 (v5 user_models)",
  "backend/infrastructure/database.py:341-353 (add_play_segment: duration = end-start)",
  "backend/infrastructure/database.py:362-383 (add_candidate_segment, confidence/stats)",
  "backend/infrastructure/database.py:427-475 (query_play_segments + filters + events join ?
the UI read path)",
  "backend/pipeline/segmenters/gemini.py:461-498 (_populate_db: metadata = shot_count,
shot_count_confidence, detector; server/receiver = None)",
  "backend/sports/badminton.py:41-54 (Gemini output schema:
start_time,end_time,shuttle_exchange_count,shot_count_confidence ? no shot_type/player/score)",
  "backend/pipeline/segmenters/motion.py:191-250 (fabricated server/receiver + random
serve/smash/foul/winner events)",
  "backend/main.py:102-105 (CompileRequest = video_id, segment_ids, output_filename)",
  "backend/main.py:342-396 (/api/compile: select by segment_ids, min_shot_count floor,
time_pairs, VideoCompiler)",
  "backend/main.py:307-328 (/api/highlights/{video_id}/{filename} download ? only compiled
reels, not source)",
  "backend/main.py:331-340 (/api/query returns play_segments)",
  "backend/pipeline/stitcher/compiler.py:80-105 (_merge_overlapping de-dupe)",
  "backend/pipeline/stitcher/compiler.py:303-413 (compile_highlights: per-clip re-encode ->
concat -c copy -> single MP4)",
  "backend/pipeline/detectors/wasb_infer.py:547-571 (extract_frames: transient WSL PNG
frames, deleted on success ? not UI thumbnails)",
  "backend/api/static/app.js:344,363-409 (UI: no video element, data.rallies bug, select
segment_ids -> compile -> download)",

```

```

"backend/config/models.py:126 (detector_score_threshold ? a config knob, NOT a per-rally
match score)",
"docs/UI_PROPOSAL.md:46,58-59 (A4 rally-review + B5 replace static UI + B6 PATCH
/api/segments correction endpoints ? proposed, NOT implemented)"
]
},
"api": {
  "endpoints": [
    {
      "method": "GET",
      "path": "/",
      "purpose": "Serve the bundled static SPA shell (backend/api/static/index.html) or a
placeholder message.",
      "response": "HTML (HTMLResponse). text/html string.",
      "auth": "None. backend/main.py:52"
    },
    {
      "method": "GET",
      "path": "/api/config",
      "purpose": "Return the live typed AppConfig the server started with (Pydantic model
dump).",
      "response": "JSON: the full AppConfig object (indexing/stitching/output/security/etc).
backend/main.py:121-123 returns global_config.",
      "auth": "None. Leaks full server config incl. security settings. backend/main.py:121"
    },
    {
      "method": "POST",
      "path": "/api/process",
      "purpose": "Submit a hash->validate->segment job for a server-local video_path. Fast-
fails bad path / missing file / unknown segmenter, then enqueues async job.",
      "response": "202 JSON {job_id, status:'queued', poll:'/api/jobs/{job_id}'}. Body model
ProcessRequest{video_path, player_pool?, max_frames?, segmenter?}. backend/main.py:240-271",
      "auth": "None. Path containment only IF security.allowed_video_root is set
(validate_input_path), else any server-readable path. backend/main.py:240"
    },
    {
      "method": "GET",
      "path": "/api/jobs/{job_id}",
      "purpose": "Poll execution state of a submitted processing job.",
      "response": "JSON {job_id, status(queued|running|waiting|done|failed), progress,
video_id, error, result(pipeline dict incl. play_segments), reports, created_at, started_at,
finished_at}. 404 if unknown. backend/main.py:274-293",
      "auth": "None. Any caller can poll any job_id (no ownership check). backend/main.py:274"
    },
    {
      "method": "POST",
      "path": "/api/query",
      "purpose": "List/filter play (rally) segments for a video_id. This is the ONLY way to
list rallies today.",
      "response": "JSON {play_segments: [ {id, video_id, start_time, end_time, duration,
server, receiver, metadata(dict), events:[{id,segment_id,timestamp,type,player,details}]} ]}.
Body QueryRequest{video_id, server?, receiver?, duration_min?, event_type?}.
backend/main.py:331-340; shape from database.py:427-475",
      "auth": "None. Any caller can query any video_id's rallies. backend/main.py:331"
    },
    {
      "method": "POST",
      "path": "/api/compile",
      "purpose": "Stitch a CUSTOM set of selected rally segment_ids into one highlight reel
(applies stitching.min_shot_count floor + paddings + watermark).",
      "response": "JSON {status:'success', filepath: server-local out path}. 404 unknown
video; 400 no matching/all-below-floor; 500 stitch fail. Body CompileRequest{video_id,
segment_ids:[int], output_filename}. backend/main.py:342-396",
      "auth": "None. Returns a server-local filepath (not the download URL).
backend/main.py:342"
    }
  ]
}

```

```

    },
    {
        "method": "GET",
        "path": "/api/highlights/{video_id}/{filename}",
        "purpose": "Stream/download a previously compiled highlight reel by bare filename.",
        "response": "Binary FileResponse (video/mp4 or video/quicktime). 400 bad/traversal name;
404 unknown video or no compiled file. backend/main.py:307-328",
        "auth": "None. Path-traversal guarded (safe_output_filename + resolve_under_root).
backend/main.py:307"
    },
    {
        "method": "POST",
        "path": "/api/upload/init",
        "purpose": "Begin a chunked browser upload (validates extension + total-size cap).",
        "response": "JSON {upload_id, max_part_mb, next_part:'/api/upload/{id}/part/0'}. 400 bad
ext/name; 413 too big. Body UploadInitRequest{filename, total_size_bytes?}.
backend/api/uploads.py:64-86",
        "auth": "None. Upload state is in-process, global (not per-user).
backend/api/uploads.py:64"
    },
    {
        "method": "PUT",
        "path": "/api/upload/{upload_id}/part/{index}",
        "purpose": "Append one strictly-sequential raw-bytes part to an in-flight upload
(enforces per-part + total caps).",
        "response": "JSON {upload_id, received_parts, received_bytes}. 404 unknown id; 409 out-
of-order; 413 overflow (aborts). backend/api/uploads.py:89-120",
        "auth": "None. backend/api/uploads.py:89"
    },
    {
        "method": "POST",
        "path": "/api/upload/{upload_id}/complete",
        "purpose": "Finalize a chunked upload: verify part count, move into upload_dir, compute
video_id.",
        "response": "JSON {path(abs server path), video_id(MD5), size_bytes, next:'POST
/api/process with this path'}. 404 unknown id; 409 part-count mismatch. Body
UploadCompleteRequest{total_parts}. backend/api/uploads.py:123-144",
        "auth": "None. backend/api/uploads.py:123"
    },
    {
        "method": "DELETE",
        "path": "/api/upload/{upload_id}",
        "purpose": "Abort an in-flight upload and delete its partial file.",
        "response": "JSON {status:'aborted', upload_id}. 404 unknown id.
backend/api/uploads.py:147-155",
        "auth": "None. backend/api/uploads.py:147"
    }
],
"auth_model": "NONE today. Every endpoint is fully unauthenticated: there is no
Depends/Security dependency, no HTTPBearer, no API-key/token check, no DEV_USER_EMAIL read, and
no per-user scoping in any handler (verified across backend/**/*.py). CORS is wide-open:
allow_origins=['*'] with allow_credentials=False (backend/main.py:40-46). The DB schema is auth-
READY but unwired: videos.user_id and candidate_segments.user_id columns + partial indexes exist
(database.py:221-236, migration v4) and a per-user user_models table exists
(database.py:240-283, v5), but NO endpoint reads or writes user_id. INTENDED model (docs, not
built): alpha = Cloudflare Access in front of both hostnames; FastAPI trusts the 'Cf-Access-
Authenticated-User-Email' header (verify the Access JWT) for per-user scoping
(HOSTING_PLAN.md:37-38). Local-dev fallback DEV_USER_EMAIL env + an owner_email column + per-
query scoping + narrowed CORS + rate limit + per-user quota are all scoped to the UNSTARTED PR-3
(UI_ALPHA_EXECUTION_PLAN.md:32). Beta graduates to Firebase Auth (HOSTING_PLAN.md:62-63). Net:
ship-blocking gap for any multi-tenant/hosted use; fine only for the single-box local alpha it
currently is.",
"gaps": [
    "NO 'list videos / list matches' endpoint at all. There is no GET /api/videos and no DB
method to enumerate videos (no get_all_videos/list_videos in database.py; only

```

get_video(video_id) at database.py:418 which requires you to ALREADY know the id). An interactive UI cannot populate a match list without first knowing every video_id out-of-band. UI_PROPOSAL screen A1 'Matches' has no backing endpoint.",

"NO GET endpoint that returns rallies/segments for a video as JSON. The only way to list rallies is POST /api/query (backend/main.py:331), which is a body-based RPC, not a RESTful GET /api/videos/{video_id}/rallies. Functionally usable for per-rally selection (it returns id, start_time, end_time, duration, metadata, events), but not cacheable/linkable and shaped as a filter-query, not a resource listing.",

"NO per-rally thumbnail or frame endpoint. There is no GET /api/videos/{video_id}/rallies/{id}/thumbnail and no frame-serving route (grep for /thumbnail|/frame|/proxy = none). A per-rally-selection grid UI cannot show preview stills; no frame-extraction handler exists.",

"NO endpoint to serve/stream the SOURCE video or a low-res proxy for scrubbing. Only the COMPILED reel is streamable (GET /api/highlights/{video_id}/{filename}, backend/main.py:307). A rally-review UI that lets a user seek/preview each rally in the original footage has nothing to load (UI_ALPHA_EXECUTION_PLAN M3 'player seek' is unbacked).",

"Custom-selection -> reel EXISTS (POST /api/compile with arbitrary segment_ids, backend/main.py:342), but it does NOT return a download URL ? it returns a server-local filepath ('filepath': out_path, backend/main.py:394). The client must separately construct GET /api/highlights/{video_id}/{output_filename}. A cleaner API would return the ready download URL.",

"NO correction / accept-reject / boundary-nudge endpoint. There is no PATCH /api/segments/{id}; the candidate_segments.human_label/notes columns exist but no route writes them. This is explicitly the unstarted PR-4 'B6 correction loop' (UI_ALPHA_EXECUTION_PLAN.md:33). An interactive per-rally UI cannot persist user corrections today.",

"NO endpoint links a completed job to a video for follow-up. GET /api/jobs/{job_id} (backend/main.py:274) returns video_id only while the job row exists; there is no GET /api/videos/{video_id}/status to re-discover processing state later, and no listing of a video's compiled reels.",

"NO /api/health endpoint, despite HOSTING_PLAN.md:42 expecting the SPA to ping one for the 'engine offline' banner. Not built.",

"Auth/ownership gap for the UI: with no auth and no user scoping, the API cannot show a per-user match list or prevent one tester from reading/compiling another's rallies (videos.user_id exists at database.py:221-236 but is never set or filtered on). PR-3 identity scoping is unstarted (UI_ALPHA_EXECUTION_PLAN.md:32)."

```

],
"file_refs": [
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\uploads.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\job_worker.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\static\\app.js",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_ALPHA_EXECUTION_PLAN.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_PROPOSAL.md"
]
},
"ui": {
  "spa_plan": {
    "stack": "React + Vite + TypeScript + Tailwind SPA, single-page, client-side router, typed API client; deployed as static hosting (Cloudflare Pages, $0) talking to the existing FastAPI backend on the owner's GPU box via a Cloudflare Tunnel at api.khelsutra.guru. Auth in alpha = Cloudflare Access email-allowlist (OTP), zero auth code; backend reads the Cf-Access-Authenticated-User-Email header for per-user scoping. CLI is NOT replaced ? SPA hits the same FastAPI surface. Stack 'stands unless owner objects' (still listed open in execution plan). SPA lives in a SEPARATE private repo `khelsutra` under web/; backend enabler PRs stay in the indexer repo. Refs: UI_PROPOSAL.md:33-37, UI_ALPHA_EXECUTION_PLAN.md:13-15;35-40, HOSTING_PLAN.md:20-34.",
    "milestones": [
      "SPA milestones (khelsutra repo web/, UI_ALPHA_EXECUTION_PLAN.md:35-40): M1 scaffold ? Vite+React+TS+Tailwind, router, typed API client, KhelSutra Guru branding.",

```

```

    "M2 ? A2 Upload (chunked + recording-guidelines checklist) + A3 Job Progress (poll).",
    "M3 ? A1 Matches + A4 Rally Review (player seek + accept/reject/nudge wired to backend
PR-4).",
    "M4 ? A5 Highlights (compile + download) + A6 Report + engine-offline banner; plus unit
tests for API client + review-state logic and a clean `npm run build`.",
    "Backend enabler PRs gating the SPA (L1, indexer repo): PR-1 B1 async job model #16
(MERGED #87); PR-2 B2+B3 chunked upload + streaming download (MERGED #99); PR-3 B4+B7 identity &
exposure (owner_email scoping, CORS, rate limit, quota) ? open; PR-4 B6 correction loop (PATCH
/api/segments/{id}) + retire static UI (B5) ? open.",
    "Phase structure (UI_ALPHA_EXECUTION_PLAN.md:19-55): G0 setup ? Phase L (local
build+test: L1 backend, L2 SPA M1-M4, L3 local E2E, GATE GL owner sign-off) ? Phase D
(deployment D1-D3, GATE GD) ? Phase R (rollout wave 1: 5 testers ? widen to 25). Beta is a
separate 4-Drop roadmap (UI_PROPOSAL.md:73-93).",
  ],
  "routes": [
    "A1 Matches (home) ? user's videos, per-video status/verdict chip, upload CTA
(UI_PROPOSAL.md:43)",
    "A2 Upload & Process ? drag-drop, inline recording-guidelines checklist, chunked upload,
auto-start processing (UI_PROPOSAL.md:44)",
    "A3 Job Progress ? stage list (validation?segmentation?AI confirm?report) with live poll
of GET /api/jobs/{id}, verdict banner (UI_PROPOSAL.md:45)",
    "A4 Rally Review (flagged ?, 'the corpus engine') ? player + rally list, click-to-see,
per-rally Accept/Reject/nudge ±0.5s, bulk 'all good' (UI_PROPOSAL.md:46)",
    "A5 Highlights ? select accepted rallies ? name reel ? compile ? download + list of past
reels (UI_PROPOSAL.md:47)",
    "A6 Report ? customer-audience report (verdict, notes, FAQ refs) + 'nerd view' toggle
for technical.md (UI_PROPOSAL.md:48)",
  ],
  "components": [
    "6-screen alpha set A1-A6 (UI_PROPOSAL.md:39-48).",
    "Rally-review widget: video player + rally list with click-to-see, accept/reject
toggles, start/end nudge control (±0.5s), bulk-accept, per-rally chips (duration, shot count +
confidence) ? backed by B6 PATCH /api/segments/{id} ? candidate_segments.human_label/notes +
provenance (UI_PROPOSAL.md:46;59, UI_ALPHA_EXECUTION_PLAN.md:33;38).",
    "Chunked uploader (?90-95 MB parts, mp4/mov allowlist, size cap) + recording-guidelines
checklist for pre-upload education (UI_PROPOSAL.md:44;55, HOSTING_PLAN.md:38-39).",
    "Job-progress poller against GET /api/jobs/{id} (UI_PROPOSAL.md:45).",
    "Highlights compiler/download via /api/compile + streaming GET
/api/highlights/{video_id}/{name} (UI_PROPOSAL.md:47;56).",
    "Report renderer with nerd-view toggle (UI_PROPOSAL.md:48).",
    "Typed API client + review-state logic (unit-tested)
(UI_ALPHA_EXECUTION_PLAN.md:36;40).",
    "Engine-offline banner driven by /api/health ping (UI_ALPHA_EXECUTION_PLAN.md:39,
HOSTING_PLAN.md:43).",
  ],
},
  "conversational_design": "IMPORTANT / honest finding: NONE of the four docs read
(UI_PROPOSAL, UI_ALPHA_EXECUTION_PLAN, HOSTING_PLAN, KHELSUTRA_PRODUCT_OVERVIEW) specify a
conversational 'ask for a reel' query feature, and there is NO mapping of a plain-English
request to TODAY's metadata (length / count / order / which game) anywhere in them. The alpha
Highlights flow (A5) is explicitly a MANUAL select-then-compile: user selects accepted rallies ?
names the reel ? compiles ? downloads (UI_PROPOSAL.md:47). Today's metadata surfaced to the user
is per-rally duration + shot count (+confidence) as chips on the Rally Review screen
(UI_PROPOSAL.md:46) and a summary of rally count + total play time
(KHELSUTRA_PRODUCT_OVERVIEW.md:14;48;65). 'Length and order' appears ONLY as a FEEDBACK prompt
to alpha testers ('Whether the reel is the right length and order for how you'd actually use
it', KHELSUTRA_PRODUCT_OVERVIEW.md:135) ? i.e. a question asked OF the user, not a query the
user can issue. Conversational QA / NL query is EXPLICITLY DEFERRED: 'no conversational QA
(deferred in roadmap)' (UI_PROPOSAL.md:66). Where a conversational layer IS specified (outside
these four docs, all future/gated): OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55 ('build the
bookkeeping NOW, punt the model' ? punt the conversational feature until extraction quality is
solid) with the queryable per-video event-index schema as 'the conversational seam' (lines 102,
160); PLATFORM_ARCHITECTURE.md:263-272 (NL queries about rallies/shots/technique, must be
localized); I18N_PLAN.md:159 (P4 'Conversational + coach-mode (future): locale-aware NL query
intake ? templated localized answers ? gated on those features existing'). So the metadata that

```

COULD back a future query (rally_ordinal, start/end ? length, shot_count, which-game/match keying) is being reserved in the event-index schema (POST_166?MILESTONES.md:55), but no length/count/order/which-game parsing of a plain-English reel request is designed in the read docs. Explicitly deferred dimensions: shot-type and player are both roadmap-only ? shot detection (smashes/drops/net/serves/clears) is 'on the roadmap, not available today' (KHELSUTRA_PRODUCT_OVERVIEW.md:116;160) and player tracking/identification is a hard non-goal / strictly-future biometrics (KHELSUTRA_PRODUCT_OVERVIEW.md:160;181, and the alpha non-goal 'no athlete profiles', UI_PROPOSAL.md:64).",

"positioning_notes": [

"Per-rally selection IS in the alpha product, but as TWO distinct surfaces: (a) Rally Review (A4) for accept/reject/boundary-nudge correction, and (b) Highlights (A5) for picking which accepted rallies go into a named reel that is compiled + downloaded. Both are manual UI interactions, not a query/builder (UI_PROPOSAL.md:46-47).",

"A 'conversational builder' is NOT positioned as a current product feature in any of the four docs. The reel-building experience shipped in alpha is manual select-then-compile (UI_PROPOSAL.md:47); conversational QA is listed as an alpha non-goal, 'deferred in roadmap' (UI_PROPOSAL.md:66).",

"The strategic framing of per-rally selection is the CORPUS-GROWTH ENGINE, not a consumer convenience: A4 Rally Review is flagged ? as 'the corpus engine' ? every accept/reject/nudge produces the labels the owned-model track is starved for, persisted to candidate_segments.human_label/notes with provenance (UI_PROPOSAL.md:9-14;46;59). This realizes the v1 'correction loop' / 'co-builders' positioning (KHELSUTRA_PRODUCT_OVERVIEW.md:92-94).",

"A future conversational surface is positioned as a downstream OWNED-MODEL / platform capability, deliberately decoupled: extract-once/query-many event index is the 'conversational seam' to be built early while the conversational *model/feature* is punted until extraction quality is high (OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55;102;160; PLATFORM_ARCHITECTURE.md:263-272). The product overview frames this as the path 'toward an AI assistant coach' ? direction, not today's buttons (KHELSUTRA_PRODUCT_OVERVIEW.md:111-126).",

"Honesty discipline is explicit in positioning: the overview twice tells users the 'gets sharper over time' / deeper-analysis story is the direction being built toward, NOT a switch already on (KHELSUTRA_PRODUCT_OVERVIEW.md:105-107;124-126)."

],

"available_today": [

"Upload a full session/match video (phone-on-tripod or GoPro; long videos fine), chunked upload (KHELSUTRA_PRODUCT_OVERVIEW.md:44;47, UI_PROPOSAL.md:44)",

"Automatic rally detection ? finds rallies + their start/end (KHELSUTRA_PRODUCT_OVERVIEW.md:45)",

"Highlight reel: rallies stitched into one downloadable/shareable MP4 with a 'Generated by Khelsutra.guru' mark (KHELSUTRA_PRODUCT_OVERVIEW.md:46)",

"Simple summary: rally count + total actual play time (KHELSUTRA_PRODUCT_OVERVIEW.md:14;48)",

"Re-use & download: process once, return later, download the reel; re-processing REPLACES the prior result (KHELSUTRA_PRODUCT_OVERVIEW.md:49;66)",

"Per-rally review + correction: click-to-see, accept/reject, boundary nudge $\pm 0.5s$, bulk-accept, with per-rally duration + shot-count(+confidence) chips (UI_PROPOSAL.md:46) ? the core human correction loop",

"Manual reel building: select accepted rallies ? name ? compile ? download (UI_PROPOSAL.md:47)",

"Customer-audience report with verdict/notes/FAQ + nerd-view toggle (UI_PROPOSAL.md:48)",

"Background/async processing ? close the tab, come back later (KHELSUTRA_PRODUCT_OVERVIEW.md:62-64; async job model PR-1 #87 merged)",

"Backend reality: alpha serves the Gemini segmenter path (no GPU in request path), ~ 0.93 F1 on clean single-court footage, $\sim \$0.05/\text{run}$ (UI_PROPOSAL.md:16-18, HOSTING_PLAN.md:32-33)",

"Invite-only, English-only private web app (KHELSUTRA_PRODUCT_OVERVIEW.md:59;156)"

],

"roadmap": [

"Shot detection ? smashes, drops, net shots, serves, clears within each rally (KHELSUTRA_PRODUCT_OVERVIEW.md:116;160) ? DEFERRED, not today",

"Deeper per-rally / per-match analysis: patterns, tendencies (KHELSUTRA_PRODUCT_OVERVIEW.md:117)",

"Per-match history / progress over weeks-months (KHELSUTRA_PRODUCT_OVERVIEW.md:118)",

"AI assistant coach ? coaching insights that support, never replace, the coach (KHELSUTRA_PRODUCT_OVERVIEW.md:119-120)",

"Conversational / NL query (the 'ask for a reel' surface): seam/event-index built now, model/feature punted until extraction quality is solid; must be localized

```

(OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55, PLATFORM_ARCHITECTURE.md:263-272, I18N_PLAN.md:159
P4) ? DEFERRED (UI_PROPOSAL.md:66)",
    "More sports: table tennis next, then other racket/court sports
(KHELSUTRA_PRODUCT_OVERVIEW.md:121; alpha is badminton-only, UI_PROPOSAL.md:65)",
    "Localization: Kannada + Hindi alongside English (KHELSUTRA_PRODUCT_OVERVIEW.md:122)",
    "Beta drops (UI_PROPOSAL.md:73-93): Drop 1 accounts/academies + real auth (Firebase/Clerk,
18+ only, roles); Drop 2 cloud storage + GPU box as pull-based byo_worker + direct-to-cloud
resumable uploads + email-when-ready; Drop 3 Review v2 (J/K/L + [/]) shortcuts, frame-step,
timeline minimap, batch accept, tags, expiring share links, per-player aggregates ? NO
gait/biometrics); Drop 4 monetization (Razorpay per-match credits) + training-data opt-in
(default OFF, adult-only, minors excluded)",
    "Per-person tracking / biometrics: explicitly NOT today and gated as strictly-future
(KHELSUTRA_PRODUCT_OVERVIEW.md:107;160;181)",
    "Pricing not set (KHELSUTRA_PRODUCT_OVERVIEW.md:162)"
  ],
  "file_refs": [
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_PROPOSAL.md (alpha 6
screens A1-A6 :39-48; A4 'corpus engine' :46; A5 manual compile :47; conversational QA deferred
:66; stack :33-37; beta drops :73-93; backend B1-B7 :52-60)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_ALPHA_EXECUTION_PLAN.md
(SPA milestones M1-M4 :35-40; backend PRs L1 :29-33; phases/gates :19-55; stack-open :15)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md (alpha
topology + Gemini path :20-34; auth header :37-38; 100MB chunk cap :38-39; offline banner :43)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\KHELSUTRA_PRODUCT_OVERVIEW.md (today vs roadmap :40-49 / :111-126; 'right length
and order' feedback prompt :135; shot/player non-goals :160;181; honesty caveats
:105-107;124-126)",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55;102;160 (conversational seam built now,
feature punt) ? NOT among the four docs read but the authoritative location of conversational
design",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PLATFORM_ARCHITECTURE.md:263-272 (NL query surface, localized) ? supporting",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\I18N_PLAN.md:159 (P4
conversational NL intake, gated) ? supporting",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:55 (event-index schema
fields rally_ordinal, start/end, shot_count) ? the metadata a future query would map onto"
  ]
},
"convention": {
  "required_sections": [
    "0. TL;DR ? 2-5 sentences: goal, approach, single most important caveat; call out any
standalone-actionable finding (template:30)",
    "1. Problem & goal ? why now; concrete user-facing outcome; what 'good' looks like; link
the code/docs a reader must read to get current (template:33). NOTE: real examples often split
this into '1. Problem & motivating example' + '2. Goal & non-goals'
(PROMINENT_COURT_DETECTION.md:14,19)",
    "2. Decisions locked ? table capturing answered questions so they aren't relitigated
(template:35)",
    "3. Foundation ? research / prior art *(OPTIONAL ? include only when the plan rests on
external facts: cited research, legal/compliance, benchmarks, prior-art; mark researched-vs-
verified; list sign-off gates)* (template:42)",
    "4. Design / architecture ? the actual plan: data contracts, diagrams, and a REUSE MAP
naming existing modules/code this builds on so new-vs-reused is clear (template:45)",
    "5. Threat model / risk table *(OPTIONAL ? security/privacy/compliance work; who/what is
protected vs NOT; residual risks stated plainly)* (template:48). In non-security docs this slot
is used for a 'Risks & limits' or 'Experiment/measurement plan' section instead
(PROMINENT_COURT_DETECTION.md:77,86)",
    "6. Honest limits ? what this does NOT do ? the false-confidence guardrail: what a green
suite / DONE status does NOT prove (template:51)",
    "7. Deliverables & progress tracker ? SOURCE OF TRUTH ? one small PR per row, branched
from origin/master, no stacking; update row status+PR as work lands (template:54)",
    "8. Open questions ? owner / external ? split blocking-gates from nice-to-knows; name who
decides (template:61)",

```

"9. Definition of done ? explicit acceptance checklist; when all ticked flip Status->DONE and archive (template:64)",

"10. References ? Internal code anchors [verified] (paths) · Internal docs · External (cited sources) (template:68)",

"Changelog (trailing, after a --- rule) ? dated bullets of what changed (template:73)"

],

"header_fields": [

"Status: a backtick-quoted lifecycle state (DRAFT | IN PROGRESS | DONE) optionally qualified, e.g. `DRAFT` ? `owner-gated for any default flip` or `IN PROGRESS` (template:21, PROMINENT_COURT_DETECTION.md:3, REQUEST_ACCESS_FORM.md:3)",

"Owner: name, e.g. Avi (template:21, PROMINENT_COURT_DETECTION.md:3)",

"Created: YYYY-MM-DD (template:21)",

"Last updated: YYYY-MM-DD (template:21)",

"Lifecycle: literal `DRAFT ? IN PROGRESS ? DONE ? archived` with the note '(move to docs/archives/ when DONE)' (template:22)",

"Tracking anchors: '\$N progress tracker is the source of truth; indexed in docs/README.md; pointer in memory current-state (and docs/NEXT_STEPS.md once work starts)' (template:23, PROMINENT_COURT_DETECTION.md:5)",

"Relation to existing docs: <extends | supersedes | peer-of ?> ? which sibling docs/gaps this attacks (template:24, PROMINENT_COURT_DETECTION.md:6)",

"Honesty note: 'mark each claim [verified] / [researched] / [design]. Not legal/financial advice where applicable.' (template:25, PROMINENT_COURT_DETECTION.md:7)"

],

"progress_tracker_format": "Section §7 titled 'Deliverables & progress tracker' marked '? source of truth' (template:54). It opens with a Legend line then a markdown table. CANONICAL legend (template:55): 'Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One small PR per row, branched from origin/master, no stacking. Update the row (status + PR link) as work lands.' CANONICAL columns (template:57-59): | ID | Deliverable | Depends on | Gated? | Status | PR | ? first row example: | P0 | ? | ? | No | ? | ? |. IDs are short prefixed codes (P0/P1?, C0/C1?, or tier-coded E1/W1/M1/R1). The Status cell carries the glyph PLUS a short measured result, e.g. '? baseline F1 0.300, merge_rate 0.523' or '? golden ?F1 +0.006 (n.s.)' (RALLY_QUALITY_RESEARCH.md:537-539). Richer variant (RALLY_QUALITY_RESEARCH.md:535) adds a 'Tier' and 'Acceptance / promote criterion' column: | ID | Deliverable | Tier | Depends on | Acceptance / promote criterion | Status |. A measured project also keeps a separate cumulative 'The number' progress table (RALLY_QUALITY_RESEARCH.md:522-531): | after | F1 | merge_rate | ? vs baseline |. Glyph note: template/PROMINENT use ? for done; RALLY_QUALITY_RESEARCH uses ? and its own legend '? not started · ? in progress · ? done' (RALLY_QUALITY_RESEARCH.md:533) ? both accepted.",

"dod_format": "A dedicated '## 9. Definition of done' section (template:64; PROMINENT_COURT_DETECTION.md:112; REQUEST_ACCESS_FORM.md:101). Format = an explicit acceptance checklist of GitHub-style checkboxes '- [] ?', each a verifiable condition (PROMINENT_COURT_DETECTION.md:113-116, REQUEST_ACCESS_FORM.md:102-106). Closing instruction in the template: 'When all ticked ? flip Status ? DONE and archive.' (template:65). For a multi-PR tracked project the DoD may instead be a numbered list of completion clauses tied to tiers/the headline number (RALLY_QUALITY_RESEARCH.md:553-560: 'The project is complete when: 1. ? 2. ? 3. ?'), optionally followed by an owner-adjustable scope note (RALLY_QUALITY_RESEARCH.md:562). Items should be measurable and may carry honesty tags like [measured].",

"style_notes": [

"Numbering: sections are numbered §0..§10 starting at '## 0. TL;DR'; §7 is always the progress tracker / source of truth; subsections use dotted numbers (§4.1, §7.1, §7.4) (template:29-69, RALLY_QUALITY_RESEARCH.md:183-560)",

"OPTIONAL sections (§3 Foundation, §5 Threat model) are included only when relevant and DELETED when they don't apply; their slots are reused for domain-appropriate sections (Risks & limits, Experiment plan) (template:14-15,42,48; PROMINENT_COURT_DETECTION.md:77,86)",

"Status header is a single multi-line blockquote ('> ') at the very top right under the H1 title, fields joined by ' · ' (template:21-25; PROMINENT_COURT_DETECTION.md:3-7; REQUEST_ACCESS_FORM.md:3-9)",

"Honesty discipline is mandatory and pervasive: tag every claim inline with backticked `[verified]` (checked vs code/measured), `[researched]` (needs sign-off), or `[design]` (proposed); never aspirational ? the doc reflects real shipped state (template:11-12,25; PROMINENT_COURT_DETECTION.md:7,14,54). Record refuted/killed claims and corrections explicitly with an 'honesty note' (RALLY_QUALITY_RESEARCH.md:96,364)",

"The doc POINTS to detail (code anchors, research artifacts, sibling docs) and does NOT duplicate it; §7 is the single source of truth (template:10,13; RALLY_QUALITY_RESEARCH.md:17)",

"Code/config references are concrete and clickable: relative markdown links to sibling

```
docs e.g. [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md), and path:line anchors to code e.g.
backend/pipeline/detectors/tracknet_runner.py:256 (RALLY_QUALITY_RESEARCH.md:79,581;
PROMINENT_COURT_DETECTION.md:57-62)",
    "Memory wikilinks [[double-bracket-slug]] cross-reference auto-memory notes, e.g.
[[launch-report-convention]], [[weak-signals-retained]], [[decision-rigor-norm]]
(PROMINENT_COURT_DETECTION.md:7,12,28)",
    "Tone: dense, engineering-precise, plainspoken, opinionated; bold for headline takeaways
and the load-bearing word; em-dashes and parentheticals for asides; uses ? for caveats and a
'Dead-ends / do-NOT' list so decisions aren't relitigated (RALLY_QUALITY_RESEARCH.md:261,490;
PROMINENT_COURT_DETECTION.md). Every signal/feature change ships flag-gated, default-OFF,
promoted only on measured lift",
    "Per-row workflow rule baked into the legend: 'One small PR per row, branched from
origin/master, no stacking' ? matches CLAUDE.md branching discipline (template:55;
PROMINENT_COURT_DETECTION.md:98; REQUEST_ACCESS_FORM.md:79)",
    "Trailing '### Changelog' after a horizontal rule (---), dated bullets newest-appended;
only present once the doc has live history (template:71-74;
PROMINENT_COURT_DETECTION.md:122-126)",
    "khelsutra-repo voice (for docs authored there): same status-header + tracker discipline
('mirrors the engine repo's PROJECT_DOC_TEMPLATE'), but tracker may slim to | ID | Deliverable |
Gated? | Status | PR |; privacy-first / contact-only-never-a-training-source guardrail echoed;
portability framed as 'adapter swap, not a rewrite'; comparison tables with two host/option
columns; runbook fenced-code blocks; 'Honest notes' section closing the doc
(REQUEST_ACCESS_FORM.md:6,80; PORTABILITY.md:2,15,44)"
  ],
  "file_refs": [
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\RALLY_QUALITY_RESEARCH.md",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\PROMINENT_COURT_DETECTION.md",
    "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\REQUEST_ACCESS_FORM.md",
    "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PORTABILITY.md"
  ]
}
}
```

THREE CANDIDATE APPROACHES:

```
[
  {
    "name": "RallyReel Timeline",
    "one_liner": "A scrubbable match-timeline surface in the alpha SPA where every engine-
detected rally is a toggleable marker over the source video; flipping markers on/off drives a
live \\N clips / M:SS\\ reel-preview footer that compiles the exact selection into one
downloadable MP4.",
    "primary_surface": "Timeline-first: a single horizontal, scrubbable match timeline spanning
the full source duration, with one clickable rally marker (start..end band) per play_segment.
The video player sits above the timeline and the seek head is shared; toggling a marker is the
primary selection gesture, and a sticky reel-preview footer (selected count + summed duration)
is the persistent call-to-action. Rally cards become a secondary, scroll-synced list ? the
timeline is canonical.",
    "data_flow": "engine (Gemini segmenter) writes play_segments rows (id, start_time, end_time,
duration, metadata.shot_count/+confidence, video_id) -> SPA loads them via POST /api/query
{video_id} which returns {play_segments:[...]} (today the ONLY list path; app.js currently mis-
reads data.rallies, a known bug to fix) -> SPA holds them in client state as RallyMarker[] keyed
by segment_id, each with a boolean `selected` (default ON) -> the timeline renders one band per
marker positioned by start_time/end_time over the full duration (duration derived as
max(end_time) since no match-duration field exists) -> user toggles markers; the footer
recomputes selectedCount + sum(end_time-start_time) live, purely client-side, no round-trip ->
on \\\"Build reel\\\" the SPA POSTs the selected segment_ids to POST /api/compile {video_id,
segment_ids, output_filename}, which cuts time_pairs from videos.filepath and writes one MP4 ->
SPA constructs the download URL itself (GET /api/highlights/{video_id}/{filename}) since
/api/compile returns only a server-local filepath. Inline per-rally PREVIEW (play just this span
in the player) requires the source video, which has NO streaming endpoint today ? so MVP
previews by seeking the player to start_time and auto-pausing at end_time, which needs a source-
video stream that does not yet exist.",
```

```

"components": [
  "TimelineCanvas ? full-duration horizontal track; one positioned band per play_segment
(left=start_time/total, width=duration/total); selected vs deselected styling; shared seek head;
click-band-to-toggle, click-track-to-see; hover tooltip shows duration +
shot_count(+confidence) from metadata.",
  "RallyMarker band ? the toggleable unit; shows selected/deselected state, duration label,
and a shot_count chip (only field that actually exists per rally today); double-click = preview
this span in the player.",
  "VideoPlayer ? <video> element bound to the source stream; programmatic seek to a rally's
start_time, auto-pause at end_time for single-rally preview; the current static UI has NO
<video> element so this is net-new. Requires a source-stream endpoint (gap).",
  "ReelPreviewFooter (sticky) ? live 'N clips / total M:SS', a min_shot_count-floor warning
when a selected rally would be dropped by stitching.min_shot_count, reel-name input, and the
'Build reel' button. Pure client-side recompute on every toggle.",
  "SelectionToolbar ? Select all / Clear / Invert / 'Only ? X shots' quick-filters computed
client-side over the loaded play_segments (no new API; mirrors the existing /api/query
duration_min filter but done client-side for instant feedback).",
  "RallyList (secondary) ? scroll-synced cards reusing the existing app.js card renderer
(with the data.rallies->play_segments bug fixed); each card mirrors its timeline band's selected
state two-way.",
  "TypedApiClient ? query(video_id), compile(video_id, segment_ids, name),
buildDownloadUrl(video_id, name); wraps the body-RPC endpoints; this is the M1 'typed API
client' milestone deliverable.",
  "JobProgressGate ? reuses GET /api/jobs/{id} poll so the timeline only mounts once
status==done and play_segments exist in the job result.",
  "useReelSelection hook ? owns RallyMarker[] + selected set + derived totals; unit-tested
(the M4 'review-state logic' test target).",
],
"api_additions": [
  "GET /api/videos/{video_id}/source (NEW, ship-blocking for inline preview/scrub) ? Range-
capable stream of videos.filepath. The real gap: only compiled reels stream today (GET
/api/highlights/{video_id}/{filename}, main.py:307-328); there is NO endpoint serving the source
video, so a timeline player has nothing to load. Smallest enabler: a Range-supporting
FileResponse over db.get_video(video_id)['filepath'], guarded like the highlights route.",
  "GET /api/videos/{video_id}/rallies (NEW, recommended, non-blocking) ? RESTful,
cacheable/linkable GET returning the same shape as POST /api/query's {play_segments:[...]}. Real
gap: listing rallies today is only the body-RPC POST /api/query (main.py:331-340), not linkable
per-video. MVP can ship on POST /api/query and add this as polish.",
  "GET /api/videos (NEW, needed to pick WHICH match) ? there is literally no list-videos
endpoint or DB method (no get_all_videos/list_videos; only get_video(id) at database.py:418
needs the id already). The timeline needs a video_id; without this the SPA must carry the id
from the just-finished job. Out of MVP scope if entered straight from A3 Job Progress, but
required for a standalone 'open a past match' entry.",
  "POST /api/compile SHOULD also return the ready download URL (ENHANCEMENT) ? today it
returns only a server-local filepath ('filepath': out_path, main.py:394) and the client
reconstructs GET /api/highlights/{video_id}/{filename} by hand (exactly what app.js does at line
392). Returning {download_url} removes that fragile client-side string-building. Non-blocking;
MVP keeps the client-built URL.",
  "GET /api/videos/{video_id}/rallies/{id}/thumbnail (NEW, roadmap only) ? no
frame/thumbnail extraction or column exists (wasb_infer extract_frames is transient WSL PNGs,
deleted on success, never keyed to rallies). NOT in MVP; timeline bands use color/label, not
stills.",
  "PATCH /api/segments/{id} (NEW, roadmap ? the deferred PR-4/B6 correction loop) ?
candidate_segments.human_label/notes/confirmed_segment_id columns EXIST (database.py:166-168)
but no route writes them. Needed only when timeline editing graduates from ephemeral selection
to persisted accept/reject/nudge corrections. Explicitly out of MVP.",
],
"mvp_scope": [
  "Fix the existing read bug: parse POST /api/query's {play_segments:[...]} (not
data.rallies) ? app.js:344 ? so rallies actually load. This alone is a prerequisite and is a
one-line truth-up.",
  "Render the timeline from play_segments: full-duration track (total = max(end_time)), one
toggleable band per segment positioned by start_time/end_time, all selected by default. No video
stream required for THIS slice ? bands + the secondary card list only.",
  "Live reel-preview footer: client-side selectedCount + sum(duration) updated on every

```

toggle; reel-name input; surface a warning for any selected rally whose metadata.shot_count is below stitching.min_shot_count (read from GET /api/config) since /api/compile will silently drop it.",

"Build reel: POST /api/compile {video_id, segment_ids: selected, output_filename}; on success construct GET /api/highlights/{video_id}/{filename} (client-side, as app.js already does) and offer the download link. This closes the loop 'pick rallies -> get a reel' with ZERO new backend endpoints.",

"Select-all / clear / invert / 'only >= X shots' selection toolbar ? pure client-side over loaded segments. No new API.",

"Enter from A3 Job Progress carrying the finished job's video_id (no list-videos needed for MVP)."

],

"roadmap_extensions": [

"Add GET /api/videos/{video_id}/source (Range stream) -> wire the VideoPlayer so a rally band double-click previews that exact [start,end] span in-page (seek + auto-pause). This is the first true 'inline preview' and the highest-value post-MVP addition.",

"Add GET /api/videos (list matches) + GET /api/videos/{video_id}/rallies -> a real A1 Matches home and linkable/refreshable timeline URLs decoupled from the job lifecycle.",

"Make /api/compile return {download_url} -> drop client-side URL string-building.",

"Persisted corrections via PATCH /api/segments/{id} (PR-4/B6): graduate timeline toggles into accept/reject + boundary-nudge writing candidate_segments.human_label/notes/confirmed_segment_id ? turning the timeline into the ? corpus-growth engine, not just an ephemeral picker.",

"Per-rally thumbnails (GET .../thumbnail) once frame extraction is keyed to rallies -> richer bands.",

"Timeline minimap, frame-step, keyboard J/K/L + [/] shortcuts, tags (the beta 'Review v2' Drop-3 set) layered onto the same timeline surface.",

"When shot-type/player/score detection ships (all roadmap-only today), color/group bands by those dimensions ? schema seams reserved in metadata/event-index, do NOT consume them until they exist."

],

"integration_with_conversational": "The timeline is the manual, direct-manipulation surface; the conversational 'ask for a reel' builder is the same selection model driven by language. They share ONE source of truth: the client-side selected-set over play_segments and the identical compile call (POST /api/compile with segment_ids). A conversational request ('a 2-minute reel of the longest rallies') resolves to a predicate over fields that EXIST today ? duration (end_time-start_time) and metadata.shot_count ? ranks/selects segment_ids, then sets the SAME timeline selection so the user SEES the proposed bands light up and can hand-correct before building. The conversational layer never compiles independently; it pre-populates the timeline's selection and footer. This is honest about scope: conversational QA is explicitly DEFERRED in the docs (UI_PROPOSAL:66) and only duration/shot_count are queryable today (shot_type/player/score are roadmap-only), so the language builder's vocabulary is bounded to length/count/longest-N until those fields land. The timeline is the visual contract that keeps the conversational builder honest and correctable.",

"pros": [

"Ships 'pick rallies -> get a reel' with ZERO new backend endpoints for the core slice ? it rides POST /api/query + POST /api/compile + GET /api/highlights, all of which exist and are verified working.",

"The dominant timeline metaphor makes selection spatial and reversible; live 'N clips / M:SS' gives instant feedback that a card list can't, directly answering the alpha feedback prompt 'is the reel the right length'.",

"Selection state is a clean client-side set that the conversational builder can reuse verbatim ? one selection model, two input methods, no divergence.",

"Honest about data: bands only show duration + shot_count(+confidence), the fields that actually exist per rally, so nothing claims shot-type/player/score that the engine can't produce.",

"Fixes the live data.rallies bug as a side effect, and the min_shot_count warning prevents the silent-drop surprise baked into /api/compile.",

"Graceful upgrade path: the same timeline becomes the corpus-engine (PR-4 PATCH) and the beta Review-v2 surface without a rewrite."

],

"cons": [

"True inline preview/scrub needs a source-video stream that does NOT exist (only compiled reels stream); without GET /api/videos/{video_id}/source the MVP timeline is select-and-build with no in-page playback ? the weakest part of a 'timeline-first' pitch.",

"Total-duration positioning has no real match-duration field; deriving total = max(end_time) leaves the timeline tail (post-last-rally) and any pre-first-rally dead air unrepresented unless a real duration is added.",

"No thumbnails means bands are abstract color/label rectangles ? less scannable than a stills grid for a casual user.",

"Selection is ephemeral (in-memory) in MVP; refresh loses it, and nothing is persisted until PATCH /api/segments lands, so it's a picker, not yet the corpus engine the docs flag as the strategic point.",

"A long match with many short rallies makes bands thin and fiddly on a single track; needs zoom/minimap (deferred to beta) to stay usable.",

"Standalone entry needs GET /api/videos (doesn't exist); MVP is tied to arriving from the Job Progress screen with the video_id in hand."

],

"honest_caveats": [

"The ONLY per-rally fields that exist today are start_time, end_time, duration, and metadata.shot_count(+shot_count_confidence) on the Gemini path; server/receiver and all events are FABRICATED random values written only by the motion segmenter ? the timeline must not surface them as real.",

"shot_type, player identity, match/point score, and game/set grouping do NOT exist in any table or detector ? they are roadmap-only and must not be consumed, filtered on, or displayed as rally attributes.",

"play_segments has NO confidence column; only candidate_segments.confidence exists and the live trajectory hybrids pass None ? so a per-rally confidence chip is only meaningful via metadata.shot_count_confidence (Gemini), not a rally-level confidence.",

"No source-video, proxy, frame, or thumbnail endpoint exists ? any in-page playback/scrub/preview is BLOCKED until a source-stream endpoint is added; the verified-working loop without new APIs is select-then-compile-then-download only.",

"/api/compile silently applies stitching.min_shot_count and drops sub-floor segments (exempting unknown counts), and returns a server-local filepath, NOT a download URL ? the client must build GET /api/highlights/{video_id}/{filename} itself (as the current app.js already does).",

"There is NO auth or per-user scoping on any endpoint and no list-videos endpoint; this design is honest only for the single-box local/invite alpha and assumes the video_id is carried in from the job, not discovered.",

"All claims here are grounded in the supplied research and spot-verified against backend/main.py:331-396, database.py:418-475 & 150-178, and app.js:340-411; per repo convention a buildable version of this should land as a tracked project doc (PROJECT_DOC_TEMPLATE) with a S7 progress tracker, default-OFF flagging for any engine-side enabler, before code."

]

},

{

"name": "Rally Grid",

"one_liner": "A responsive grid of rally cards over the engine's play_segments index, with min-length/game/sort filters, individual + bulk checkbox selection, and a sticky \"build reel\" tray that fires the existing /api/compile to produce one downloadable MP4 ? the visual twin of the conversational builder.",

"primary_surface": "Grid/list of rally cards (responsive masonry-ish card grid), with a left/top filter rail and a sticky bottom selection tray. NOT a timeline; NOT conversational. Each card = one play_segments row (index, duration, shot_count + confidence chip), a checkbox, and a thumbnail slot (placeholder until a thumbnail endpoint exists).",

"data_flow": "engine (Gemini segmenter writes play_segments at process time) -> API: SPA calls POST /api/query {video_id} to get the rally list (id, start_time, end_time, duration, metadata.shot_count/shot_count_confidence/detector, server/receiver, events) ? this is the ONLY rally-listing path today (backend/main.py:331-340) -> SPA state: normalize into a client-side RallyCard[] (id, index=array position, durationSec, shotCount, shotConf), held in a store with a Set<segment_id> selection + filter/sort settings (all client-side; query supports only duration_min/server/receiver/event_type server-side) -> reel: POST /api/compile {video_id, segment_ids: Array.from(selection), output_filename} (backend/main.py:342-396) re-encodes per-clip + concats to one MP4; SPA then builds the download URL itself GET /api/highlights/{video_id}/{output_filename} (compile returns a server-local filepath, NOT a URL ? main.py:394). Card index/duration/shot_count are the only honest, non-fabricated fields; server/receiver/events exist in the response but are random fabrications from the motion segmenter and MUST NOT be surfaced on the Gemini alpha path.",

"components": [

"RallyGrid ? responsive CSS-grid of RallyCard; virtualized if a video yields many

```

rallies",
    "RallyCard ? thumbnail slot (placeholder/poster until /thumbnail exists) + index badge +
duration + shot_count(+confidence) chip + selection checkbox + hover 'preview' affordance
(disabled until a clip/source endpoint exists)",
    "FilterRail ? min-length slider (maps to query duration_min OR client filter), sort
control (by index/duration/shot_count, client-side), and a 'game' selector rendered as a
DISABLED/'coming soon' control because no game/set index exists in the contract",
    "SelectionTray (sticky) ? running count + summed duration of selected rallies, Select-all
/ Clear, reel-name input, and Build Reel button -> /api/compile",
    "BulkBar ? select-all-visible, select-by-filter ('all > Ns'), invert, clear (operates on
the client RallyCard set)",
    "CompileResult ? post-compile state: download link (client-constructed GET /api/highlights
URL) + inline <video> preview of the COMPILED reel (the only video the API can stream today)",
    "ApiClient ? typed wrapper over POST /api/query + POST /api/compile + GET /api/highlights,
sharing the exact CompileRequest shape {video_id, segment_ids, output_filename}",
    "EmptyState/ErrorState ? 'no rallies / engine offline' (engine-offline needs /api/health
which is unbuilt)",
    "MatchPicker (header) ? needs a NEW list-videos endpoint; until then takes a video_id from
the upload/job-progress flow's result"
],
"api_additions": [
    "GET /api/videos (NEW) ? list the user's videos so the grid can be reached without knowing
a video_id out-of-band. No enumeration endpoint or DB method exists today (only
get_video(video_id) at database.py:418 requires the id). Needed for MatchPicker/A1; for MVP the
SPA can instead carry video_id forward from the upload->job result.",
    "GET /api/videos/{video_id}/rallies (NEW, optional but strongly preferred) ? a
cacheable/linkable RESTful read of the rally list. Today the only path is the body-RPC POST
/api/query (main.py:331), which is not GET-cacheable or deep-linkable. MVP can ship on POST
/api/query unchanged; this is a clean-up.",
    "Make /api/compile return the download URL (CHANGE) ? it returns a server-local filepath
('filepath': out_path, main.py:394), forcing the client to hand-build GET
/api/highlights/{video_id}/{filename}. Return {download_url} so the tray can link directly. Low-
risk additive field.",
    "GET /api/videos/{video_id}/rallies/{segment_id}/thumbnail (NEW) ? per-rally poster still
for the card. No thumbnail column, no frame endpoint, and the only frame extraction
(wasb_infer.extract_frames) is transient WSL detector input deleted on success
(wasb_infer.py:547-571). Cards ship with a placeholder until this exists; this is the single
biggest visual gap.",
    "GET /api/videos/{video_id}/clip?start=&end= OR a source-stream endpoint (NEW) ? to let a
card preview/scrub a single rally in the original footage. Only the COMPILED reel is streamable
today (GET /api/highlights, main.py:307); there is NO source-video or arbitrary-span endpoint.
Until then 'preview' can only play the compiled reel after building.",
    "GET /api/health (NEW) ? for the engine-offline banner the SPA is specified to ping
(HOSTING_PLAN.md:42); not built.",
    "Auth/user-scoping on /api/query + /api/compile + /api/videos (NEW) ? every endpoint is
unauthenticated with no user scoping; videos.user_id (database.py:221-236) exists but is never
set/filtered. Multi-tester hosting needs the unstarted PR-3 identity scoping (Cf-Access-
Authenticated-User-Email)."
],
"mvp_scope": [
    "Reach a video_id via the existing upload->job result (skip a videos-list endpoint for
v0): carry job result.video_id into the grid route.",
    "Fetch rallies with POST /api/query {video_id} (no new backend) and FIX the static UI's
data.rallies bug by reading response.play_segments (app.js:344).",
    "Render RallyGrid of RallyCards showing ONLY honest fields: index (array order), duration
(end-start), and shot_count + shot_count_confidence chip from metadata; thumbnail = a
placeholder tile (no thumbnail endpoint yet).",
    "Client-side filters/sort: min-length slider, sort by duration/shot_count/index ? all in
the browser over the fetched list (server-side query only supports duration_min honestly on the
Gemini path).",
    "Individual + bulk checkbox selection backed by a Set<segment_id>; Select-all-visible /
Clear.",
    "Sticky SelectionTray with live count + summed duration and a reel-name field.",
    "Build Reel -> POST /api/compile {video_id, segment_ids, output_filename}; surface the
min_shot_count-floor 400 honestly (some selected rallies may be dropped by

```

```

stitching.min_shot_count; unknown counts are exempt ? main.py:362-383).",
  "On success, construct GET /api/highlights/{video_id}/{output_filename} client-side, show
a download link AND an inline <video> of the compiled reel (the only streamable video today).",
],
"roadmap_extensions": [
  "Per-rally thumbnails once GET .../thumbnail lands ? replace placeholder tiles with real
posters (turns the grid from a list into a true visual picker).",
  "Hover/click single-rally preview once a clip/source-span endpoint exists ? scrub a rally
in original footage before selecting.",
  "Game/set grouping + a real 'which game' filter when a game/set index is detected
(roadmap-only today: no column, no detection ? contract confirms absent).",
  "Shot-type and player filters/badges when shot detection + player ID ship (both roadmap-
only; events.type/player on the Gemini path are empty, motion's are fabricated ? do NOT wire).",
  "Sort/badge by confirmed-rally confidence once play_segments gains a confidence column
(only candidate_segments.confidence exists today and trajectory hybrids pass None).",
  "Inline Accept/Reject/nudge on each card (the A4 'corpus engine') once PATCH
/api/segments/{id} (PR-4, B6) writes candidate_segments.human_label/notes ? merges the selection
grid with the correction loop.",
  "Compile returns download_url directly; add /api/health-driven engine-offline banner; add
GET /api/videos for a true MatchPicker home (A1).",
  "Save/reuse named selections + list past compiled reels per video (needs a 'list a video's
reels' endpoint, also absent).",
],
"integration_with_conversational": "[\"Shared selection state is the seam: the
conversational 'ask for a reel' builder and the grid both write the SAME Set<segment_id>; a chat
request ('all rallies over 20s') just applies a filter+selection on the grid, and the user can
then eyeball/tweak the checkboxes before Build Reel. Both compile through the identical POST
/api/compile {video_id, segment_ids, output_filename}.\", \"Grid as the 'show your work' surface
for the conversational layer: NL queries map only onto fields that exist today (length via
duration, count via shot_count, order via index) ? the grid makes that mapping visible and
correctable, which is exactly the honesty discipline the docs demand (conversational QA is
explicitly deferred, UI_PROPOSAL.md:66).\", \"One API client, two front-ends: the conversational
builder produces a segment_ids set; the grid renders/edits it. No new backend is needed for the
pairing beyond what each surface already needs.\", \"Honest divergence: the conversational
builder is roadmap/deferred and must localize; the grid is buildable on TODAY's API. The grid
can ship first and de-risk the compile/selection contract the conversational builder will
reuse.\"]",
"pros": [
  "Buildable today on the real API: POST /api/query (list) + POST /api/compile (build) + GET
/api/highlights (download) cover the full pick->reel loop with ZERO new backend for the MVP.",
  "Directly delivers the owner's framing ? dynamic per-video selection, not a fixed
threshold ? by replacing the silent min_shot_count cut with explicit user checkboxes.",
  "Bulk + filter selection scales to long match videos far better than reviewing rallies
one-by-one (the conversational/single-card surfaces do not).",
  "Reuses the existing compile contract exactly (segment_ids), so it composes cleanly with
the conversational builder and the future correction loop.",
  "Honest by construction: only surfaces index/duration/shot_count ? the three non-
fabricated fields ? and hides server/receiver/events that are random on the motion path.",
  "Fixes a known real bug (app.js reads data.rallies; API returns play_segments) as part of
the rebuild.",
  "A natural home for the A4 'corpus engine' accept/reject later: a card grid is where
inline correction controls live.",
],
"cons": [
  "Cards are visually thin until a thumbnail endpoint exists ? a 'grid' of placeholder tiles
is just a styled list; the headline visual payoff is gated on NEW backend (frame extraction).",
  "No per-rally preview before selecting (no source/clip-span endpoint) ? users pick partly
blind; they can only preview the COMPILED reel after building, then re-select if wrong.",
  "Server-side filtering is weak: /api/query honestly supports only duration_min on the
Gemini path (server/receiver/event_type are motion-only fabrications), so most filtering/sorting
must be client-side over the full list.",
  "No videos-list or per-user scoping endpoint means the grid can't stand alone as a home
screen yet and can't be safely multi-tenant (unstarted PR-3).",
  "'Which game' / shot-type / player filters that users will expect from a sports grid
simply have no data ? risk of promising filters the engine can't back.",

```

```

    "compile applies min_shot_count silently-ish; the UI must explain why some checked rallies
    didn't make the reel, or it looks broken.",
    "Grid + conversational + (future) A4 review is three overlapping reel-building surfaces ?
    needs deliberate IA so they don't confuse alpha testers."
  ],
  "honest_caveats": [
    "No thumbnail/frame endpoint, no per-rally source/clip-span endpoint, no videos-list
    endpoint, no /api/health, and no auth/user-scoping exist today ? all are NEW work; the MVP
    deliberately ships without them (placeholder tiles, carry video_id from upload, single-box
    unauth).",
    "shot_type, player identity, match/point score, and game/set index are ALL roadmap-only
    per the contract ? none have a column or any detection on the served Gemini path.
    server/receiver/events ARE returned by /api/query but are random fabrications from the motion
    segmenter; this design must NOT surface them as real.",
    "play_segments has no confidence column; metadata.shot_count_confidence is a Gemini-
    provided string (often 'Unknown'), not a numeric confidence ? render it as-is, don't compute a
    meter.",
    "/api/compile returns a server-local filepath, not a URL; the client must construct the
    GET /api/highlights download URL itself (current app.js already does this) until compile is
    changed to return download_url.",
    "The min_shot_count floor can drop user-selected rallies at compile (unknown counts
    exempt); this is real engine behavior (main.py:362-383) the UI must surface, not hide.",
    "The conversational 'ask for a reel' builder is explicitly DEFERRED in the read docs
    (UI_PROPOSAL.md:66); 'pairs with conversational' here means a shared segment_ids/compile
    contract, not a shipped second surface.",
    "This is a design proposal [design]; code anchors cited are [verified] against the engine
    repo, but nothing here is implemented in khelsutra web/ (currently only web/README.md)."
  ]
},
{
  "name": "RallyPicker ? conversational-seeded, manually-tuned reel builder",
  "one_liner": "A plain-English query box (deterministic filter/sort over the metadata the
  engine already returns: duration, count, order, shot_count) pre-selects a rally set; the user
  then fine-tunes by toggling individual rally cards, and one Compile call stitches the selection
  into a downloadable reel ? zero new engine intelligence required for MVP.",
  "primary_surface": "Hybrid: a conversational query bar sitting ATOP a selectable rally
  list/grid. The query is the primary entry and writes the selection set + sort order; the list is
  the fine-tune surface where the user overrides per-rally. Both drive one shared selection state
  (a Set of play_segments.id) that feeds POST /api/compile. Not a timeline-scrub UI (no
  proxy/source-stream exists to scrub against).",
  "data_flow": "engine: Gemini segmenter -> play_segments rows (id, start_time, end_time,
  duration, metadata.shot_count/shot_count_confidence/detector) [verified database.py:122-135,
  gemini.py:461-498]. API: SPA calls POST /api/query {video_id} ->
  {play_segments:[{id,start_time,end_time,duration,server,receiver,metadata,events}]} [verified
  main.py:331-340, database.py:427-475] (the ONLY rally-list path today; server/receiver/events
  are motion-only fabrications and are NOT shown). SPA state: rallies[] held in a typed client
  store; a deterministic parseQuery() compiles the English string into a {filter, sort} predicate
  over ONLY {duration, ordinal/order, shot_count} and applies it to produce
  selectedIds:Set<number> + displayOrder. User toggles/bulk-edits mutate selectedIds. reel: on
  Compile the SPA sorts selectedIds back into start_time order (the stitcher de-dupes/merges by
  time anyway via _merge_overlapping [verified compiler.py:80-105]), POSTs /api/compile {video_id,
  segment_ids, output_filename} -> {status,filepath} [verified main.py:342-396], then constructs
  and fetches GET /api/highlights/{video_id}/{output_filename} [verified main.py:307-328] to
  download/preview the single compiled MP4 (the only playable artifact that exists today ? no
  source/proxy/per-span stream).",
  "components": [
    "QueryBar ? text input + example chips ('rallies over 10s, longest first', 'top 10
    longest', 'all rallies, in order', '20+ shots'); shows a parsed-pill readout ('filter:
    duration>10s · sort: duration desc') so the deterministic interpretation is transparent and
    editable.",
    "parseQuery() ? pure client-side, deterministic, UNIT-TESTED grammar -> {predicate,
    comparator}; understands ONLY length (seconds/minutes), count (top N / first N), order
    (chronological / longest-first / shortest-first), and shot_count thresholds. Explicitly
    refuses/greys-out shot-type/player/score tokens with an inline 'not available yet' note.",
    "RallyList/Grid ? one selectable card per play_segment: checkbox, ordinal (#1..#n by

```

```

start_time), mm:ss start, duration badge, shot_count + confidence chip (only when
metadata.shot_count present), 'included/excluded' state driven by selectedIds.",
"SelectionBar ? live count + total selected play-time (sum of duration), 'Select all' /
'Clear' / 'Invert', and the running output-name field.",
"useRallySelection() ? shared selection-state hook (selectedIds:Set, displayOrder,
apply(queryResult), toggle(id), bulk ops); UNIT-TESTED, mirrors the M4 'review-state logic' the
execution plan already calls for.",
"Typed API client ? query(videoId), compile(videoId, segmentIds, name) returning a ready
download URL it derives itself (closes the 'compile returns a path not a URL' gap client-side),
getRallies via the GET shim if/when added.",
"CompileButton + ReelResult ? disabled when selectedIds empty; surfaces the
min_shot_count-floor caveat (segments below stitching.min_shot_count are silently dropped
server-side [verified main.py:362-381]); on success shows a <video src=GET /api/highlights/...>
preview + download link.",
"EngineOfflineBanner ? pings GET /api/health (NEW) to gate the UI, matching
HOSTING_PLAN.md:43 expectation."
],
"api_additions": [
"[REAL GAP, P1] GET /api/videos/{video_id}/rallies -> 200 {play_segments:[...]} ? a
cacheable/linkable RESTful read alias for the body-RPC POST /api/query (gap: 'no GET endpoint
that returns rallies'; main.py:331). MVP can ship on POST /api/query as-is; this is a thin
wrapper around the existing db.query_play_segments, no new data.",
"[REAL GAP, P1] Make POST /api/compile return the download URL, not just a server-local
path ? add 'download_url': '/api/highlights/{video_id}/{output_filename}' to the response (gap:
'compile does NOT return a download URL'; main.py:394). Until then the SPA derives it client-
side from the bare filename.",
"[REAL GAP, P1] GET /api/health -> 200 {status:'ok'} ? does not exist; HOSTING_PLAN.md:43
expects it for the engine-offline banner (gap list item).",
"[REAL GAP, P2] GET /api/videos (list videos for the current user) -> backs the match
picker so the UI can resolve a video_id without knowing it out-of-band (gap: 'NO list-videos
endpoint / no get_all_videos in database.py'; needs a new db enumerate + the unstarted PR-3 user
scoping). Out of MVP scope but required to reach a usable multi-video flow.",
"[REAL GAP, ROADMAP] PATCH /api/segments/{id} (accept/reject/nudge) writing
candidate_segments.human_label/notes ? the dead correction-loop columns exist but no route
writes them (gap; unstarted PR-4, UI_PROPOSAL B6). NOT needed for pick->reel; deferred to the A4
corpus-engine track.",
"[REAL GAP, ROADMAP] A per-[start,end] clip or low-res proxy stream endpoint + a source-
video stream ? neither exists (playback.notes: only compiled reels are streamable). Needed for
true per-rally preview/scrub; MVP previews the compiled reel only."
],
"mvp_scope": [
"Single known video_id (resolved from the just-finished job's result.video_id [verified
jobs.result holds play_segments] or pasted) ? NO match-list dependency, sidestepping the missing
GET /api/videos.",
"Fetch rallies via the EXISTING POST /api/query {video_id} (no new backend code to
start).",
"Render the selectable RallyList with ordinal + mm:ss + duration + shot_count/confidence
chips (all fields the payload already carries).",
"QueryBar with deterministic parseQuery() covering exactly four dimensions that exist
today: duration threshold, count (top/first N), order (chronological/longest/shortest),
shot_count threshold; with a visible parsed-pill readout and example chips. No LLM.",
"Toggle individual rallies + Select-all/Clear/Invert; live selected-count and total-play-
time.",
"Name + POST /api/compile {video_id, selectedIds, name}; on success build the GET
/api/highlights/{video_id}/{name} URL client-side, preview in a <video>, and offer download.",
"Surface the min_shot_count-floor caveat and an empty/all-dropped error state from
compile's 400.",
"Unit tests for parseQuery() and useRallySelection() (matches the execution plan's M4 'API
client + review-state logic' test bar).",
],
"roadmap_extensions": [
"Add GET /api/health + EngineOfflineBanner (small, real gap).",
"Swap POST /api/query for GET /api/videos/{video_id}/rallies and have compile return
download_url (both real gaps; makes the flow RESTful/cacheable).",
"Add GET /api/videos + PR-3 identity scoping -> a real multi-video Match picker (A1)

```

```

instead of single-id.",
    "Wire A4 correction loop via PATCH /api/segments/{id} (PR-4) so accept/reject/nudge
persist to human_label/notes ? feeds the owned-model corpus AND lets the picker exclude
'rejected' rallies.",
    "Per-rally preview: add a per-[start,end] clip or low-res proxy + source-stream endpoint
so query results can be previewed before compiling (today only the final reel previews).",
    "Grow the query grammar to shot_type / player / score ONLY when those fields actually land
(roadmap-only today); keep parseQuery() the single seam so the conversational model can later
replace the deterministic parser without UI change (the 'conversational seam' framing,
OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55).",
    "Promote parseQuery() to a localized NL intake (Kannada/Hindi) per I18N_PLAN P4 once the
dimensions are stable."
],
    "integration_with_conversational": "The conversational 'ask for a reel' builder and this
manual picker are ONE surface, not two: the query bar is the conversational entry and it WRITES
INTO the same selection state the manual toggles mutate. parseQuery() is deliberately
deterministic (filter/sort over existing metadata) ? it is NOT the deferred LLM conversational-
QA feature (explicitly deferred, UI_PROPOSAL.md:66) and needs no unavailable data. It IS,
however, the exact 'conversational seam' the owned-model plan reserves
(OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55;102): when shot/player/score extraction later lands,
the deterministic parser can be upgraded to a localized NL model behind the same
{predicate,comparator} interface with zero change to the list/compile UI. This also unifies with
the alpha's two manual surfaces: A5 Highlights (select->name->compile) becomes 'query-seed then
fine-tune', and the A4 Rally-Review accept/reject state (when PR-4 lands) becomes an additional
default filter ('exclude rejected') the query composes with.",
    "pros": [
        "MVP needs ZERO new engine endpoints ? it runs entirely on the existing POST /api/query +
POST /api/compile + GET /api/highlights contract [all verified].",
        "Every query dimension (length/count/order/shot_count) maps to a field the payload already
returns; nothing aspirational is consumed.",
        "Deterministic parser = predictable, testable, offline, no API cost, no hallucination ?
and the visible parsed-pill keeps the interpretation honest and correctable.",
        "Conversational-first without an LLM: satisfies the owner's 'pairs with the ask-for-a-reel
builder' intent while honoring 'conversational QA deferred'.",
        "One shared selection state means the manual fine-tune and the query are never out of
sync; the stitcher's own _merge_overlapping makes selection-order mistakes harmless.",
        "Cleanly forward-compatible: parseQuery() is the single seam to later drop in the real
conversational/owned model and new fields, 'adapter swap not a rewrite'."
    ],
    "cons": [
        "No per-rally preview in MVP ? there is no source/proxy/per-span stream (real gap), so the
user picks from text metadata (duration, shot_count) and only previews the FINAL compiled reel.
Mitigation requires a new clip/proxy endpoint.",
        "Single-video MVP leans on knowing the video_id (from the finishing job) because GET
/api/videos doesn't exist; a true match list is a follow-on.",
        "compile returns a server path not a URL today, so the SPA must reconstruct the download
URL from the bare filename (brittle until the small backend fix lands).",
        "The min_shot_count server-side floor can silently drop selected rallies; the UI must
explain this or the selection won't match the reel.",
        "shot_count itself is Gemini's shuttle_exchange_count (with a fallback heuristic when
absent) [gemini.py:470-485], so a 'by shot count' query is only as good as that estimate ?
honest-label it as approximate."
    ],
    "honest_caveats": [
        "NO conversational LLM is in this MVP and none is needed; parseQuery() is a deterministic
string->{filter,sort} compiler. Calling it 'conversational' must not imply NL understanding
beyond the four hard-coded dimensions.",
        "shot_type, player identity, and match/game score do NOT exist in any table or detection
today (server/receiver/events are motion-only RANDOM fabrications; score is absent everywhere) ?
they are excluded from the query grammar and must be greyed-out, not faked. [verified contract
rally_fields_roadmap]",
        "No proxy/thumbnail/source-stream/per-span endpoint exists ? per-rally visual preview is
genuinely unbuildable today without new engine API; MVP previews only the compiled reel.
[verified playback section]",
        "The whole flow is unauthenticated and unscoped today (CORS '*', no user_id read) ? fine

```

for the single-box local alpha it targets, a ship-blocker for any multi-tenant host; out of this feature's scope but stated plainly. [verified auth_model]",

"A green SPA build + passing parseQuery/selection unit tests prove the picker logic, NOT that the engine's rally boundaries are correct ? rally-detection quality (esp. the measured rally-END weakness) is a separate track and unaffected by this UI.",

"This is a [design] proposal grounded in [verified] code anchors; it has not been built or run. Per repo convention it should be written up as a tracked project doc (PROJECT_DOC_TEMPLATE) before implementation, and any default-config behavior touching compile follows the Launch Report convention."

```
]
}
```

Tasks:

1. Score each approach 1-5 on fit_to_data (matches the real contract), mvp_shippability, core_ux (the pick-rallies->reel job), extensibility (to roadmap shot/player). Sum to total.
2. Recommend an architecture ? most likely a HYBRID that grafts the best of each (e.g. a grid/list as the workhorse, a conversational query bar that pre-selects, and a compact timeline/preview of the chosen set). Justify with the scores.
3. Define a crisp MVP (smallest slice delivering pick->reel), then phased milestones.
4. List concrete api_additions, each tagged new/exists/modify against the REAL gaps in the research. Do NOT invent capabilities the engine lacks.
5. Surface genuinely owner-gated open_decisions (product/scope/positioning) and risks.
6. Produce design_markdown: a full tracked-design-doc BODY following the project's PROJECT_DOC_TEMPLATE structure found in the research (status header fields, decisions-locked, numbered sections covering: problem & positioning, what the engine gives us today [grounded], the design/architecture, data flow & API additions, MVP & phased milestones, honest scope today-vs-roadmap, open decisions, risks, progress tracker, definition-of-done). Match the khelsutra doc voice. Be HONEST: scope queries to length/count/order/game only; shot/player are roadmap. Cite the grounding refs inline where you assert an engine capability. Return the schema.

2. assistant (2026-06-20T08:05:24.465Z)

I'll review the key grounding references to verify the contract claims before consolidating, then produce the recommendation and design doc.

3. user (2026-06-20T08:05:25.901Z)

```
331     @app.post("/api/query")
332     def query_play_segments(req: QueryRequest):
333         filters = {
334             "server": req.server,
335             "receiver": req.receiver,
336             "duration_min": req.duration_min,
337             "event_type": req.event_type
338         }
339         segments = db_instance.query_play_segments(req.video_id, filters)
340         return {"play_segments": segments}
341
342     @app.post("/api/compile")
343     def compile_highlights(req: CompileRequest):
344         video = db_instance.get_video(req.video_id)
345         if not video:
346             raise HTTPException(status_code=404, detail="Indexed video record not found.")
347
348         # Retrieve matching play segments from db
349         all_segments = db_instance.query_play_segments(req.video_id, {})
350         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
351
352         if not matched_segments:
353             raise HTTPException(status_code=400, detail="No matching play segments found to stitch.")
354
355         # Reject path traversal / absolute output names (os.path.join would otherwise let an
```

```

356     # absolute or '..'-laden output_filename write anywhere on disk).
357     try:
358         out_name = safe_output_filename(req.output_filename)
359     except ValueError as e:
360         raise HTTPException(status_code=400, detail=str(e)) from e
361
362     # The configured shot-count floor (stitching.min_shot_count) applies on the API
363     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
364     # metadata are exempt: the floor filters known counts only, so explicitly
365     # requested manual/legacy segments can't silently vanish under the default floor.
366     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
367     kept = []
368     for s in matched_segments:
369         meta = s.get("metadata") or {}
370         sc = meta.get("shot_count") if isinstance(meta, dict) else None
371         try:
372             if sc is not None and int(sc) < stitching.min_shot_count:
373                 continue
374             except (TypeError, ValueError):
375                 pass # unparseable count -> treat as unknown, keep
376             kept.append(s)
377     if len(kept) < len(matched_segments):
378         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
379         dropped "
380         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
381         segments below the floor.")
382         if not kept:
383             raise HTTPException(status_code=400,
384                                 detail=f"All {len(matched_segments)} matching segments fall
385             below "
386             f"stitching.min_shot_count={stitching.min_shot_count}.")
387
388     # Extract start/end time pairs
389     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
390
391     # Run compiler with the configured stitching paddings + optional branding watermark
392     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
393     or "output"
394     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
395                             end_padding=stitching.end_padding,
396                             watermark=stitching.watermark)
397     try:
398         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
399         return {"status": "success", "filepath": out_path}
400     except Exception as e:
401         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
402         {str(e)}") from e
403
404     # --- CLI Debug Utility & Orchestration ---
405     def run_cli_diagnose_clip(video_path: str, timestamp: float):
406         """CLI debugging utility to examine segment properties around a timestamp."""

```

4. user (2026-06-20T08:05:26.790Z)

```

1     # Request-Access form ? segmented lead capture + setup discovery
2
3     > **Status:** `IN PROGRESS` . **Owner:** Avi . **Created:** 2026-06-20 . **Last
4     updated:** 2026-06-20
5     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
6     > **Relation:** first tracked work item in the `khelsutra` repo; mirrors the engine
7     repo's
8     > `PROJECT_DOC_TEMPLATE` discipline (status header . decisions-locked . $6 tracker .
9     DoD).

```

```

7      > Feeds the platform direction in the engine repo's `PLATFORM_ARCHITECTURE.md §3`
(StorageBackend /
8      > ComputeBackend) and `VIDEO_LOCALITY_MODEL.md` ? see §7.
9
10     ---
11
12     ## 0. TL;DR
13     Replace the landing page's `mailto:` "Request access" CTA with a real form that (a)
segments
14     **academies vs individual enthusiasts**, (b) asks the two questions that actually decide
how we'd
15     serve a user ? ***"do you have a GPU machine?"** (compute placement) and ***"where do your
videos
16     live?"** (storage backend) ? and (c) keeps a free-text message. Submissions are stored
in our own
17     **Cloudflare D1** via a small **Worker** (`POST /api/request-access`). The form doubles
as PMF +
18     architecture research.
19
20     ## 1. Problem & goal
21     Today the CTA is `mailto:hello@khelsutra.guru` ? zero structure, nothing collected, no
segmentation.
22     We want **structured, queryable** interest signals from two distinct audiences, captured
the moment
23     someone raises a hand, **without shipping lead data to a third party** (privacy-first
ethos).
24     **Goal:** a clean on-brand form whose responses land in our own Cloudflare account,
queryable later.
25
26     ## 2. Decisions locked
27     - **Backend = Cloudflare Worker + D1** (owner decision, 2026-06-20). Data stays in our
Cloudflare
28     account; queryable; no third party. (Alternatives considered: hosted form
(Tally/Formspre
29     third-party data), Worker?email-only (no DB), structured `mailto` (no collection).
Rejected for
30     the reasons in parens.)
31     - **Two audience segments:** `academy` and `individual` ? the form lightly adapts
(academy also
32     asks club name + #courts).
33     - **The two discovery questions are first-class** (not optional polish): GPU/compute +
storage ?
34     they map 1:1 onto the BYO-compute / BYO-storage architecture forks (§7).
35     - **Free-text message retained** ("Anything else?").
36     - **Privacy:** store only what's submitted + a coarse country (from
`request.cf.country`); **no raw
37     IP**; consent checkbox for "email me about the alpha"; this data is
**operations/contact only**,
38     never a training source (same guardrail as the engine repo).
39     - **Deploy model changes:** the site moves from dashboard direct-upload ? **Wrangler**
(Worker with
40     static assets + D1 binding). Documented in §5; owner runs the `wrangler` deploy (our
account/auth).

```

5. assistant (2026-06-20T08:05:31.283Z)

I have enough verified grounding. Let me check the doc template structure quickly to match voice.

6. user (2026-06-20T08:05:31.515Z)

```

75     > Until the Worker ships, the form works via the `mailto` fallback (re-upload `site/` as
today).
76     > Git?Cloudflare auto-deploy still intentionally not connected.
77

```

```

78     ## 6. Deliverables & progress tracker ? source of truth
79     Legend: ? Todo · ? In progress · ? Done. One PR per row off `origin/master`.
80     | ID | Deliverable | Gated? | Status | PR |
81     |----|-----|-----|-----|----|
82     | P0 | This design doc + `NEXT_STEPS.md` | No | ? | #2 |
83     | P1 | Form UI (segments + GPU + storage + free text + consent) w/ `mailto` fallback |
No | ? | #2 |
84     | P2 | Worker + D1 + `wrangler.toml` + `schema.sql` + **unit tests (10 green)** | owner
deploy | ? | this PR (built+tested; owner runs `wrangler d1 create` + `deploy`) |
85     | P3 | Anti-spam (Turnstile) + per-submission email notify | No | ? | ? |
86     | P4 | Tiny admin/export view (auth'd) to read responses | owner | ? | ? |
87
88     ## 7. Why this matters beyond lead capture
89     The two questions are **the architecture forks** from the engine repo's BYO analysis:
90     - **GPU?** ? compute placement: desktop/local (their GPU) vs hosted vs zero-infra-BYO
91     (`PLATFORM_ARCHITECTURE.md §3.2`, `DESKTOP_APP_PLAN.md` P0 spike).
92     - **Storage?** ? which `StorageBackend` + whether autosync is worth building
93     (`PLATFORM_ARCHITECTURE.md §3.1`, the cloud-storage/autosync thread).
94     So early responses directly inform what to build next ? not just who to email.
95
96     ## 8. Open questions
97     1. Turnstile now or wait for spam? (Default: honeypot now, Turnstile if needed.)
98     2. Email-notify per submission, or just query D1 weekly? (Default: notify in P2.)
99     3. Do we localize the form (Kannada/Hindi) for academies now or at beta? (Default:
English now.)
100
101     ## 9. Definition of done
102     - [ ] Form replaces the `mailto` CTA, segments academy/individual, asks GPU + storage +
free text.
103     - [ ] Submissions persist to Cloudflare D1 via the Worker; honeypot drops bots.
104     - [ ] Privacy posture honored (no raw IP; consent; contact-only).
105     - [ ] Deployed live on khelsutra.guru; a test submission appears in D1.
106     - [ ] Doc tracker + DoD updated; archived when P1?P3 land.
107

```

7. assistant (2026-06-20T08:05:35.304Z)

I have everything I need. The contract is verified against main.py (query/compile/min_shot_count floor/filepath-not-URL all confirmed), and I have the khelsutra doc voice. Producing the consolidated recommendation.

8. user (2026-06-20T08:07:59.216Z)

Structured output provided successfully

9. assistant (2026-06-20T08:08:10.322Z)

Consolidated recommendation delivered via StructuredOutput.

Summary of the decision:

```

**Scores (fit_to_data / mvp_shippability / core UX / extensibility ? total):**
- RallyPicker (conversational-seeded): 5 / 5 / 4 / 5 ? **19** (winner)
- Rally Grid: 4 / 4 / 3 / 4 ? **15**
- RallyReel Timeline: 3 / 3 / 4 / 4 ? **14**

```

Recommendation: a HYBRID ? selectable rally **list/grid** as the workhorse (Rally Grid), a **deterministic query bar** as fast pre-select (RallyPicker), one shared `Set<segment_id>` feeding the existing `POST /api/compile`, with the **timeline + per-rally preview deferred** to a later phase because today's contract cannot stream source video. All three candidates already converge on the same seam (one selected-set, one compile call), so fusing them costs no architectural compromise ? it only sequences which input mode ships first.

Key honesty constraints baked in (all verified against `backend/main.py`):

- MVP needs ****zero new engine endpoints**** ? rides ``POST /api/query`` + ``POST /api/compile`` + ``GET /api/highlights``.
- Queries scoped to ****length / count / order / shot_count only****; shot-type/player/score are roadmap-only with no column or detector.
- ``server`/`receiver`/`events`` are motion-only random fabrications and must stay hidden.
- The ``stitching.min_shot_count`` floor silently drops selected rallies (``main.py:362-383``) ? surfaced as a UI warning.
- ``/api/compile`` returns a server-local filepath, not a URL ? client builds the download URL.

The full design-doc body (status header, decisions-locked table, numbered sections, S7 tracker, DoD, grounded code anchors) is in the ``design_markdown`` field, matching the khelsutra ``REQUEST_ACCESS_FORM.md`` voice. I did not write any files per instructions.